```
library(DBI)
library(bigrquery)

project_id <- "YOUR_PROJECT"
dataset_id <- "YOUR_DATASET"
billing_project <- "YOUR_BILLING_PROJECT"

con <- dbConnect(
  bigquery(),
  project = project_id,
  dataset = dataset_id,
  billing = billing_project
)
```

1-

```
sql_month_keys <- "
CREATE OR REPLACE TABLE
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_month_keys_25m`

PARTITION BY feature_month
CLUSTER BY party_id

AS

SELECT
    s.party_id,

    DATE(s.observation_date) AS observation_date,

    DATE(s.first_qualifying_alert_date)
       AS first_qualifying_alert_date,

    CAST(s.label_3m AS INT64) AS label_3m,

    month_offset,

    LAST_DAY(
       DATE_SUB(
          DATE_TRUNC(
             DATE(s.observation_date),
             MONTH
          ),
          INTERVAL month_offset MONTH
       ),
```

```
    MONTH
  ) AS feature_month,

  CASE
    WHEN month_offset = 24 THEN 1
    ELSE 0
  END AS seed_month_flag

FROM
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023` s

CROSS JOIN
  UNNEST(GENERATE_ARRAY(0, 24)) AS month_offset
"

DBI::dbExecute(con, sql_month_keys)
```

2-

```
sql_step2 <- "
CREATE OR REPLACE TABLE
 `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_trades_25m`

PARTITION BY feature_month
CLUSTER BY party_id, acct_id

AS

SELECT

  k.party_id,
  k.observation_date,
  k.first_qualifying_alert_date,
  k.label_3m,

  k.month_offset,
  k.feature_month,
  k.seed_month_flag,

  t.trades_seq_num,
  t.acct_id,
  t.acct_num,

  t.acct_typ_cde,
  t.porf_typ_cde,
```

```
    t.kob_posn_1_cde,
    t.kob_posn_2_cde,

    CAST(t.open_dte AS DATE) AS open_dte,
    CAST(t.clsd_dte AS DATE) AS clsd_dte,
    CAST(t.paid_out_dte AS DATE) AS paid_out_dte,
    CAST(t.eff_dte AS DATE) AS eff_dte,

    t.acct_mop_cde,

    t.curr_bal_amt,
    t.past_due_amt,
    t.cr_lmt_amt,

    t.high_cr_amt,

    t.payt_amt,
    t.payt_due_amt,
    CAST(t.payt_dte AS DATE) AS payt_dte,

    t.max_delq_amt,
    CAST(t.max_delq_dte AS DATE) AS max_delq_dte,
    t.max_delq_rte_cde,

    CAST(t.frst_delq_dte AS DATE) AS frst_delq_dte,
    CAST(t.frst_rpt_dte AS DATE) AS frst_rpt_dte,

    CAST(t.data_date AS DATE) AS source_data_date

FROM
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_month_keys_25m` k

INNER JOIN
  `YOUR_PROJECT.YOUR_SOURCE_DATASET.trades` t

    ON k.party_id = t.party_id
   AND CAST(t.data_date AS DATE) = k.feature_month
"

DBI::dbExecute(con, sql_step2)
```

3-

```
sql_step3 <- "
```

```
CREATE OR REPLACE TABLE
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_mop_transitions_25m`

PARTITION BY feature_month
CLUSTER BY party_id, acct_id

AS

WITH base AS (

    SELECT

        t.*,

        LAG(acct_mop_cde) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY feature_month
        ) AS previous_mop,

        LAG(eff_dte) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY feature_month
        ) AS previous_eff_dte,

        LAG(curr_bal_amt) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY feature_month
        ) AS previous_balance,

        LAG(cr_lmt_amt) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY feature_month
        ) AS previous_credit_limit,

        LAG(past_due_amt) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY feature_month
        ) AS previous_past_due

    FROM
      `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_trades_25m` t
),

mapped AS (

    SELECT
        *,
```

```
    CASE
      WHEN previous_mop = '01' THEN 0
      WHEN previous_mop = '02' THEN 1
      WHEN previous_mop = '03' THEN 2
      WHEN previous_mop = '04' THEN 3
      WHEN previous_mop = '05' THEN 4
      WHEN previous_mop = '06' THEN 5
      ELSE NULL
    END AS previous_delinquency_level,

    CASE
      WHEN acct_mop_cde = '01' THEN 0
      WHEN acct_mop_cde = '02' THEN 1
      WHEN acct_mop_cde = '03' THEN 2
      WHEN acct_mop_cde = '04' THEN 3
      WHEN acct_mop_cde = '05' THEN 4
      WHEN acct_mop_cde = '06' THEN 5
      ELSE NULL
    END AS current_delinquency_level

  FROM base
)

SELECT

  *,

  /* ============================================================
     Active account - original definition
     ============================================================ */

  CASE
    WHEN
      (open_dte IS NULL OR open_dte <= feature_month)

    AND
      (clsd_dte IS NULL OR clsd_dte > feature_month)

    THEN 1
    ELSE 0
  END AS active_account_flag,


  /* ============================================================
     Effective update - original definition
     ============================================================ */

  CASE
```

```
    WHEN previous_eff_dte IS NOT NULL
     AND eff_dte <> previous_eff_dte
    THEN 1
    ELSE 0
END AS effective_update_flag,


/* ============================================================
   DPD deterioration
   ============================================================ */

CASE
    WHEN previous_delinquency_level IS NOT NULL
     AND current_delinquency_level IS NOT NULL
     AND current_delinquency_level >
         previous_delinquency_level
    THEN 1
    ELSE 0
END AS dpd_deterioration_flag,


/* ============================================================
   DPD improvement
   ============================================================ */

CASE
    WHEN previous_delinquency_level IS NOT NULL
     AND current_delinquency_level IS NOT NULL
     AND current_delinquency_level <
         previous_delinquency_level
    THEN 1
    ELSE 0
END AS dpd_improvement_flag,


/* ============================================================
   Magnitude of ordinal MOP change
   ============================================================ */

CASE
    WHEN previous_delinquency_level IS NOT NULL
     AND current_delinquency_level IS NOT NULL

    THEN
       current_delinquency_level
       - previous_delinquency_level

    ELSE NULL
```

```
END AS dpd_level_change,


/* ============================================================
   Entered active adverse
   ============================================================ */

CASE
    WHEN acct_mop_cde IN ('09','9B','8A','8B')

     AND COALESCE(previous_mop,'')
         NOT IN ('09','9B','8A','8B')

    THEN 1
    ELSE 0
END AS entered_active_adverse_flag,


/* ============================================================
   Adverse resolved
   ============================================================ */

CASE
    WHEN acct_mop_cde IN ('9P','8P')

     AND COALESCE(previous_mop,'')
         <> acct_mop_cde

    THEN 1
    ELSE 0
END AS adverse_resolved_flag,


/* ============================================================
   Became unrated
   ============================================================ */

CASE
    WHEN acct_mop_cde = 'UR'
     AND COALESCE(previous_mop,'') <> 'UR'
    THEN 1
    ELSE 0
END AS became_unrated_flag,


/* ============================================================
   Left unrated
   ============================================================ */
```

```
    CASE
      WHEN previous_mop = 'UR'
       AND acct_mop_cde <> 'UR'
      THEN 1
      ELSE 0
    END AS left_unrated_flag

FROM mapped
"

DBI::dbExecute(con, sql_step3)
```

4-

```
sql_step4 <- "
CREATE OR REPLACE TABLE
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_transition_monthly`

PARTITION BY feature_month
CLUSTER BY party_id

AS

WITH agg AS (

    SELECT

        party_id,
        observation_date,
        feature_month,
        month_offset,
        seed_month_flag,


        /* Effective account updates */

        COUNT(
          DISTINCT CASE
            WHEN effective_update_flag = 1
            THEN acct_id
          END
        ) AS effectively_updated_accounts,


        /* Deterioration */
```

```
SUM(
   CASE
      WHEN dpd_deterioration_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
   END
) AS deterioration_count,


/* Improvement */

SUM(
   CASE
      WHEN dpd_improvement_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
   END
) AS improvement_count,


/* Entered adverse */

SUM(
   CASE
      WHEN entered_active_adverse_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
   END
) AS adverse_entry_count,


/* Adverse resolved */

SUM(
   CASE
      WHEN adverse_resolved_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
   END
) AS adverse_resolved_count,


/* Became unrated */
```

```
  SUM(
    CASE
      WHEN became_unrated_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
    END
  ) AS became_unrated_count,


  /* Left unrated */

  SUM(
    CASE
      WHEN left_unrated_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
    END
  ) AS left_unrated_count,


  /* Maximum ordinal deterioration */

  MAX(
    CASE
      WHEN dpd_deterioration_flag = 1
       AND effective_update_flag = 1
      THEN dpd_level_change
    END
  ) AS max_dpd_jump,


  /* Total deterioration magnitude */

  SUM(
    CASE
      WHEN dpd_deterioration_flag = 1
       AND effective_update_flag = 1
      THEN dpd_level_change
      ELSE 0
    END
  ) AS total_dpd_deterioration_points

FROM
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_mop_transitions_25m`
```

```
    GROUP BY
        party_id,
        observation_date,
        feature_month,
        month_offset,
        seed_month_flag
)

SELECT

    k.party_id,
    k.observation_date,
    k.first_qualifying_alert_date,
    k.label_3m,

    k.month_offset,
    k.feature_month,
    k.seed_month_flag,

    COALESCE(a.effectively_updated_accounts,0)
        AS effectively_updated_accounts,

    COALESCE(a.deterioration_count,0)
        AS deterioration_count,

    COALESCE(a.improvement_count,0)
        AS improvement_count,

    COALESCE(a.adverse_entry_count,0)
        AS adverse_entry_count,

    COALESCE(a.adverse_resolved_count,0)
        AS adverse_resolved_count,

    COALESCE(a.became_unrated_count,0)
        AS became_unrated_count,

    COALESCE(a.left_unrated_count,0)
        AS left_unrated_count,

    COALESCE(a.max_dpd_jump,0)
        AS max_dpd_jump,

    COALESCE(a.total_dpd_deterioration_points,0)
        AS total_dpd_deterioration_points

FROM
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_month_keys_25m` k
```

```
LEFT JOIN agg a

    ON k.party_id = a.party_id
   AND k.observation_date = a.observation_date
   AND k.feature_month = a.feature_month
"

DBI::dbExecute(con, sql_step4)
```

5A-

```
sql_step5a <- "
CREATE OR REPLACE TABLE
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_account_month_base`

PARTITION BY feature_month
CLUSTER BY party_id, acct_id

AS

SELECT

    *,

    /* Rated active account */

    CASE
        WHEN active_account_flag = 1
         AND acct_mop_cde IS NOT NULL
         AND acct_mop_cde <> 'UR'
        THEN 1
        ELSE 0
    END AS rated_active_account_flag,


    /* Delinquent active account */

    CASE
        WHEN active_account_flag = 1
         AND acct_mop_cde IN ('02','03','04','05','06')
        THEN 1
        ELSE 0
    END AS delinquent_active_account_flag,
```

```
/* 60+ */

CASE
   WHEN active_account_flag = 1
    AND acct_mop_cde IN ('03','04','05','06')
   THEN 1
   ELSE 0
END AS dpd_60plus_account_flag,


/* 90+ */

CASE
   WHEN active_account_flag = 1
    AND acct_mop_cde IN ('04','05','06')
   THEN 1
   ELSE 0
END AS dpd_90plus_account_flag,


/* Revolving / line-of-credit */

CASE
   WHEN active_account_flag = 1
    AND porf_typ_cde IN ('R','C')
   THEN 1
   ELSE 0
END AS revolving_account_flag,


/* Utilization eligible */

CASE
   WHEN active_account_flag = 1
    AND porf_typ_cde IN ('R','C')
    AND cr_lmt_amt > 0
   THEN 1
   ELSE 0
END AS utilization_eligible_flag,


/* Account-level utilization */

CASE
   WHEN active_account_flag = 1
    AND porf_typ_cde IN ('R','C')
    AND cr_lmt_amt > 0
```

```
      THEN SAFE_DIVIDE(
         COALESCE(curr_bal_amt,0),
         cr_lmt_amt
      )

      ELSE NULL
   END AS account_utilization

FROM
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_mop_transitions_25m`
"

DBI::dbExecute(con, sql_step5a)
```

5B-

```
sql_step5b <- "
CREATE OR REPLACE TABLE
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_account_behavior_monthly`

PARTITION BY feature_month
CLUSTER BY party_id

AS

WITH account_agg AS (

   SELECT

      party_id,
      observation_date,
      feature_month,
      month_offset,
      seed_month_flag,


      /* ====================================================
         Portfolio
         ==================================================== */

      COUNT(DISTINCT acct_id)
         AS reported_accounts,

      SUM(active_account_flag)
         AS active_accounts,
```

```
SUM(rated_active_account_flag)
   AS rated_active_accounts,

SUM(
   CASE
      WHEN clsd_dte IS NOT NULL
       AND clsd_dte <= feature_month
      THEN 1
      ELSE 0
   END
) AS closed_accounts,


/* ========================================================
  Exposure
  ======================================================== */

SUM(
   CASE
      WHEN active_account_flag = 1
      THEN COALESCE(curr_bal_amt,0)
      ELSE 0
   END
) AS total_active_balance,

SUM(
   CASE
      WHEN active_account_flag = 1
      THEN COALESCE(past_due_amt,0)
      ELSE 0
   END
) AS total_past_due_amount,


/* ========================================================
  Delinquency
  ======================================================== */

SUM(delinquent_active_account_flag)
   AS delinquent_active_accounts,

SUM(dpd_60plus_account_flag)
   AS dpd_60plus_accounts,

SUM(dpd_90plus_account_flag)
   AS dpd_90plus_accounts,
```

```
/* ========================================================
   Revolving - original logic
   ======================================================== */

SUM(
   CASE
      WHEN revolving_account_flag = 1
      THEN COALESCE(curr_bal_amt,0)
      ELSE 0
   END
) AS revolving_balance,

SUM(
   CASE
      WHEN revolving_account_flag = 1
      THEN COALESCE(cr_lmt_amt,0)
      ELSE 0
   END
) AS revolving_credit_limit,


/* ========================================================
   Utilization eligible universe
   active + R/C + credit limit > 0
   ======================================================== */

SUM(utilization_eligible_flag)
   AS utilization_eligible_accounts,

AVG(account_utilization)
   AS avg_utilization,

SUM(
   CASE
      WHEN utilization_eligible_flag = 1
      THEN COALESCE(curr_bal_amt,0)
      ELSE 0
   END
) AS utilization_eligible_balance,

SUM(
   CASE
      WHEN utilization_eligible_flag = 1
      THEN cr_lmt_amt
      ELSE 0
   END
) AS utilization_eligible_credit_limit
```

```
  FROM
    `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_account_month_base`

  GROUP BY
    party_id,
    observation_date,
    feature_month,
    month_offset,
    seed_month_flag
),

derived AS (

  SELECT

    *,

    /* Average balance among active accounts */

    SAFE_DIVIDE(
      total_active_balance,
      active_accounts
    ) AS avg_balance_active,


    /* Average past due among active accounts */

    SAFE_DIVIDE(
      total_past_due_amount,
      active_accounts
    ) AS avg_past_due_active,


    /* Delinquency share */

    SAFE_DIVIDE(
      delinquent_active_accounts,
      active_accounts
    ) AS delinquent_account_share,


    /* Original revolving utilization */

    SAFE_DIVIDE(
      revolving_balance,
      revolving_credit_limit
    ) AS revolving_utilization,
```

```
    /* Weighted utilization on eligible universe */

    SAFE_DIVIDE(
      utilization_eligible_balance,
      utilization_eligible_credit_limit
    ) AS weighted_utilization

  FROM account_agg
)

SELECT *
FROM derived
"

DBI::dbExecute(con, sql_step5b)
```

6-

```
sql_step6 <- "
CREATE OR REPLACE TABLE
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_behavior_monthly`

PARTITION BY feature_month
CLUSTER BY party_id

AS

SELECT

  k.party_id,
  k.observation_date,
  k.first_qualifying_alert_date,
  k.label_3m,

  k.month_offset,
  k.feature_month,
  k.seed_month_flag,


  /* ============================================================
     Portfolio
     ============================================================ */

  COALESCE(a.reported_accounts,0)
```

```
    AS reported_accounts,

COALESCE(a.active_accounts,0)
    AS active_accounts,

COALESCE(a.rated_active_accounts,0)
    AS rated_active_accounts,

COALESCE(a.closed_accounts,0)
    AS closed_accounts,


/* ============================================================
   Exposure
   ============================================================ */

COALESCE(a.total_active_balance,0)
    AS total_active_balance,

a.avg_balance_active,

COALESCE(a.total_past_due_amount,0)
    AS total_past_due_active,

a.avg_past_due_active,


/* ============================================================
   Delinquency
   ============================================================ */

COALESCE(a.delinquent_active_accounts,0)
    AS delinquent_active_accounts,

COALESCE(a.dpd_60plus_accounts,0)
    AS dpd_60plus_accounts,

COALESCE(a.dpd_90plus_accounts,0)
    AS dpd_90plus_accounts,

a.delinquent_account_share,


/* ============================================================
   Utilization
   ============================================================ */

COALESCE(a.utilization_eligible_accounts,0)
```

```
    AS utilization_eligible_accounts,

  a.avg_utilization,

  a.weighted_utilization,

  a.revolving_balance,
  a.revolving_credit_limit,
  a.revolving_utilization,


  /* ========================================================
    Behavioral transitions
    ======================================================== */

  COALESCE(t.effectively_updated_accounts,0)
    AS effectively_updated_accounts,

  COALESCE(t.deterioration_count,0)
    AS deterioration_count,

  COALESCE(t.improvement_count,0)
    AS improvement_count,

  COALESCE(t.adverse_entry_count,0)
    AS adverse_entry_count,

  COALESCE(t.adverse_resolved_count,0)
    AS adverse_resolved_count,

  COALESCE(t.became_unrated_count,0)
    AS became_unrated_count,

  COALESCE(t.left_unrated_count,0)
    AS left_unrated_count,

  COALESCE(t.max_dpd_jump,0)
    AS max_dpd_jump,

  COALESCE(t.total_dpd_deterioration_points,0)
    AS total_dpd_deterioration_points


FROM
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_month_keys_25m` k

LEFT JOIN
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_account_behavior_monthly` a
```

```
   ON k.party_id = a.party_id
  AND k.observation_date = a.observation_date
  AND k.feature_month = a.feature_month


LEFT JOIN
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_transition_monthly` t

   ON k.party_id = t.party_id
  AND k.observation_date = t.observation_date
  AND k.feature_month = t.feature_month
"

DBI::dbExecute(con, sql_step6)
```

7-

```
sql_step7 <- "
CREATE OR REPLACE TABLE
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_window_summary`

CLUSTER BY party_id

AS

WITH windows AS (

   SELECT 24 AS window_months, 23 AS max_offset
   UNION ALL
   SELECT 12, 11
   UNION ALL
   SELECT 6, 5
   UNION ALL
   SELECT 3, 2
),

expanded AS (

   SELECT

      b.*,
      w.window_months

   FROM
     `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_behavior_monthly` b
```

```
    CROSS JOIN windows w

    WHERE b.month_offset BETWEEN 0 AND w.max_offset

)

SELECT

    party_id,
    observation_date,
    first_qualifying_alert_date,
    label_3m,

    window_months,


    /* ============================================================
       PORTFOLIO
       ============================================================ */

    AVG(active_accounts)
        AS active_accounts_mean,

    STDDEV_SAMP(active_accounts)
        AS active_accounts_sd,

    AVG(reported_accounts)
        AS reported_accounts_mean,

    STDDEV_SAMP(reported_accounts)
        AS reported_accounts_sd,

    AVG(rated_active_accounts)
        AS rated_active_accounts_mean,

    STDDEV_SAMP(rated_active_accounts)
        AS rated_active_accounts_sd,


    /* ============================================================
       BALANCE
       ============================================================ */

    AVG(avg_balance_active)
        AS avg_balance_active_mean,

    STDDEV_SAMP(avg_balance_active)
        AS avg_balance_active_sd,
```

```
COUNT(avg_balance_active)
   AS balance_months_available,

AVG(total_active_balance)
   AS total_active_balance_mean,


/* ============================================================
  PAST DUE
  ============================================================ */

AVG(avg_past_due_active)
   AS avg_past_due_active_mean,

STDDEV_SAMP(avg_past_due_active)
   AS avg_past_due_active_sd,

COUNT(avg_past_due_active)
   AS past_due_months_available,

AVG(total_past_due_active)
   AS total_past_due_active_mean,


/* ============================================================
  DELINQUENCY
  ============================================================ */

AVG(delinquent_account_share)
   AS delinquent_account_share_mean,

STDDEV_SAMP(delinquent_account_share)
   AS delinquent_account_share_sd,

COUNT(delinquent_account_share)
   AS delinquency_months_available,

AVG(delinquent_active_accounts)
   AS delinquent_active_accounts_mean,

AVG(dpd_60plus_accounts)
   AS dpd_60plus_accounts_mean,

AVG(dpd_90plus_accounts)
   AS dpd_90plus_accounts_mean,
```

```
/* ============================================================
   AVG UTILIZATION
   ============================================================ */

AVG(avg_utilization)
    AS avg_utilization_mean,

STDDEV_SAMP(avg_utilization)
    AS avg_utilization_sd,

COUNT(avg_utilization)
    AS avg_utilization_months_available,


/* ============================================================
   WEIGHTED UTILIZATION
   ============================================================ */

AVG(weighted_utilization)
    AS weighted_utilization_mean,

STDDEV_SAMP(weighted_utilization)
    AS weighted_utilization_sd,

COUNT(weighted_utilization)
    AS weighted_utilization_months_available,


/* ============================================================
   REVOLVING UTILIZATION - original monthly definition
   ============================================================ */

AVG(revolving_utilization)
    AS revolving_utilization_mean,

STDDEV_SAMP(revolving_utilization)
    AS revolving_utilization_sd,

COUNT(revolving_utilization)
    AS revolving_utilization_months_available,


/* ============================================================
   DETERIORATION
   ============================================================ */

SUM(deterioration_count)
    AS deterioration_count_total,
```

```
AVG(deterioration_count)
    AS deterioration_count_avg_per_month,


/* ============================================================
   IMPROVEMENT
   ============================================================ */

SUM(improvement_count)
    AS improvement_count_total,

AVG(improvement_count)
    AS improvement_count_avg_per_month,


/* ============================================================
   ADVERSE ENTRY
   ============================================================ */

SUM(adverse_entry_count)
    AS adverse_entry_count_total,

AVG(adverse_entry_count)
    AS adverse_entry_count_avg_per_month,


/* ============================================================
   ADVERSE RESOLVED
   ============================================================ */

SUM(adverse_resolved_count)
    AS adverse_resolved_count_total,

AVG(adverse_resolved_count)
    AS adverse_resolved_count_avg_per_month,


/* ============================================================
   UNRATED
   ============================================================ */

SUM(became_unrated_count)
    AS became_unrated_count_total,

AVG(became_unrated_count)
    AS became_unrated_count_avg_per_month,
```

```
    SUM(left_unrated_count)
      AS left_unrated_count_total,

    AVG(left_unrated_count)
      AS left_unrated_count_avg_per_month,


    /* ============================================================
       EFFECTIVE UPDATES
       ============================================================ */

    SUM(effectively_updated_accounts)
      AS effectively_updated_accounts_total,

    AVG(effectively_updated_accounts)
      AS effectively_updated_accounts_avg_per_month,


    /* ============================================================
       DPD SEVERITY
       ============================================================ */

    MAX(max_dpd_jump)
      AS max_dpd_jump_window,

    SUM(total_dpd_deterioration_points)
      AS total_dpd_deterioration_points,


    /* ============================================================
       Coverage
       ============================================================ */

    COUNT(*) AS calendar_months,

    COUNTIF(reported_accounts > 0)
      AS months_with_reported_accounts,

    COUNTIF(active_accounts > 0)
      AS months_with_active_accounts

FROM expanded

GROUP BY
    party_id,
    observation_date,
    first_qualifying_alert_date,
    label_3m,
```

```
  window_months
"

DBI::dbExecute(con, sql_step7)
```

8-

```
sql_step8 <- "
CREATE OR REPLACE TABLE
 `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_window_features_wide`

CLUSTER BY party_id

AS

SELECT

  party_id,

  ANY_VALUE(observation_date) AS observation_date,

  ANY_VALUE(first_qualifying_alert_date)
    AS first_qualifying_alert_date,

  ANY_VALUE(label_3m) AS label_3m,


  /* ========================================================
    ACTIVE ACCOUNTS
    ======================================================== */

  MAX(IF(window_months = 24,
    active_accounts_mean, NULL))
    AS active_accounts_mean_24m,

  MAX(IF(window_months = 12,
    active_accounts_mean, NULL))
    AS active_accounts_mean_12m,

  MAX(IF(window_months = 6,
    active_accounts_mean, NULL))
```

```
    AS active_accounts_mean_6m,

MAX(IF(window_months = 3,
    active_accounts_mean, NULL))
    AS active_accounts_mean_3m,


MAX(IF(window_months = 24,
    active_accounts_sd, NULL))
    AS active_accounts_sd_24m,

MAX(IF(window_months = 12,
    active_accounts_sd, NULL))
    AS active_accounts_sd_12m,

MAX(IF(window_months = 6,
    active_accounts_sd, NULL))
    AS active_accounts_sd_6m,

MAX(IF(window_months = 3,
    active_accounts_sd, NULL))
    AS active_accounts_sd_3m,


/* ============================================================
   REPORTED ACCOUNTS
   ============================================================ */

MAX(IF(window_months = 24,
    reported_accounts_mean, NULL))
    AS reported_accounts_mean_24m,

MAX(IF(window_months = 12,
    reported_accounts_mean, NULL))
    AS reported_accounts_mean_12m,

MAX(IF(window_months = 6,
    reported_accounts_mean, NULL))
    AS reported_accounts_mean_6m,

MAX(IF(window_months = 3,
    reported_accounts_mean, NULL))
    AS reported_accounts_mean_3m,


/* ============================================================
   RATED ACTIVE ACCOUNTS
   ============================================================ */
```

```
MAX(IF(window_months = 24,
   rated_active_accounts_mean, NULL))
   AS rated_active_accounts_mean_24m,

MAX(IF(window_months = 12,
   rated_active_accounts_mean, NULL))
   AS rated_active_accounts_mean_12m,

MAX(IF(window_months = 6,
   rated_active_accounts_mean, NULL))
   AS rated_active_accounts_mean_6m,

MAX(IF(window_months = 3,
   rated_active_accounts_mean, NULL))
   AS rated_active_accounts_mean_3m,


/* ============================================================
  BALANCE
  ============================================================ */

MAX(IF(window_months = 24,
   avg_balance_active_mean, NULL))
   AS avg_balance_active_mean_24m,

MAX(IF(window_months = 12,
   avg_balance_active_mean, NULL))
   AS avg_balance_active_mean_12m,

MAX(IF(window_months = 6,
   avg_balance_active_mean, NULL))
   AS avg_balance_active_mean_6m,

MAX(IF(window_months = 3,
   avg_balance_active_mean, NULL))
   AS avg_balance_active_mean_3m,


MAX(IF(window_months = 24,
   avg_balance_active_sd, NULL))
   AS avg_balance_active_sd_24m,

MAX(IF(window_months = 12,
   avg_balance_active_sd, NULL))
   AS avg_balance_active_sd_12m,

MAX(IF(window_months = 6,
```

```
    avg_balance_active_sd, NULL))
    AS avg_balance_active_sd_6m,

MAX(IF(window_months = 3,
    avg_balance_active_sd, NULL))
    AS avg_balance_active_sd_3m,


MAX(IF(window_months = 24,
    balance_months_available, NULL))
    AS balance_months_available_24m,

MAX(IF(window_months = 12,
    balance_months_available, NULL))
    AS balance_months_available_12m,

MAX(IF(window_months = 6,
    balance_months_available, NULL))
    AS balance_months_available_6m,

MAX(IF(window_months = 3,
    balance_months_available, NULL))
    AS balance_months_available_3m,


/* ============================================================
   PAST DUE
   ============================================================ */

MAX(IF(window_months = 24,
    avg_past_due_active_mean, NULL))
    AS avg_past_due_active_mean_24m,

MAX(IF(window_months = 12,
    avg_past_due_active_mean, NULL))
    AS avg_past_due_active_mean_12m,

MAX(IF(window_months = 6,
    avg_past_due_active_mean, NULL))
    AS avg_past_due_active_mean_6m,

MAX(IF(window_months = 3,
    avg_past_due_active_mean, NULL))
    AS avg_past_due_active_mean_3m,


MAX(IF(window_months = 24,
    avg_past_due_active_sd, NULL))
```

```
    AS avg_past_due_active_sd_24m,

MAX(IF(window_months = 12,
    avg_past_due_active_sd, NULL))
    AS avg_past_due_active_sd_12m,

MAX(IF(window_months = 6,
    avg_past_due_active_sd, NULL))
    AS avg_past_due_active_sd_6m,

MAX(IF(window_months = 3,
    avg_past_due_active_sd, NULL))
    AS avg_past_due_active_sd_3m,


/* ============================================================
   DELINQUENCY SHARE
   ============================================================ */

MAX(IF(window_months = 24,
    delinquent_account_share_mean, NULL))
    AS delinquent_account_share_mean_24m,

MAX(IF(window_months = 12,
    delinquent_account_share_mean, NULL))
    AS delinquent_account_share_mean_12m,

MAX(IF(window_months = 6,
    delinquent_account_share_mean, NULL))
    AS delinquent_account_share_mean_6m,

MAX(IF(window_months = 3,
    delinquent_account_share_mean, NULL))
    AS delinquent_account_share_mean_3m,


MAX(IF(window_months = 24,
    delinquent_account_share_sd, NULL))
    AS delinquent_account_share_sd_24m,

MAX(IF(window_months = 12,
    delinquent_account_share_sd, NULL))
    AS delinquent_account_share_sd_12m,

MAX(IF(window_months = 6,
    delinquent_account_share_sd, NULL))
    AS delinquent_account_share_sd_6m,
```

```
MAX(IF(window_months = 3,
    delinquent_account_share_sd, NULL))
    AS delinquent_account_share_sd_3m,


/* ============================================================
   AVG UTILIZATION
   ============================================================ */

MAX(IF(window_months = 24,
    avg_utilization_mean, NULL))
    AS avg_utilization_mean_24m,

MAX(IF(window_months = 12,
    avg_utilization_mean, NULL))
    AS avg_utilization_mean_12m,

MAX(IF(window_months = 6,
    avg_utilization_mean, NULL))
    AS avg_utilization_mean_6m,

MAX(IF(window_months = 3,
    avg_utilization_mean, NULL))
    AS avg_utilization_mean_3m,


MAX(IF(window_months = 24,
    avg_utilization_sd, NULL))
    AS avg_utilization_sd_24m,

MAX(IF(window_months = 12,
    avg_utilization_sd, NULL))
    AS avg_utilization_sd_12m,

MAX(IF(window_months = 6,
    avg_utilization_sd, NULL))
    AS avg_utilization_sd_6m,

MAX(IF(window_months = 3,
    avg_utilization_sd, NULL))
    AS avg_utilization_sd_3m,


/* ============================================================
   WEIGHTED UTILIZATION
   ============================================================ */

MAX(IF(window_months = 24,
```

```
    weighted_utilization_mean, NULL))
    AS weighted_utilization_mean_24m,

MAX(IF(window_months = 12,
    weighted_utilization_mean, NULL))
    AS weighted_utilization_mean_12m,

MAX(IF(window_months = 6,
    weighted_utilization_mean, NULL))
    AS weighted_utilization_mean_6m,

MAX(IF(window_months = 3,
    weighted_utilization_mean, NULL))
    AS weighted_utilization_mean_3m,


MAX(IF(window_months = 24,
    weighted_utilization_sd, NULL))
    AS weighted_utilization_sd_24m,

MAX(IF(window_months = 12,
    weighted_utilization_sd, NULL))
    AS weighted_utilization_sd_12m,

MAX(IF(window_months = 6,
    weighted_utilization_sd, NULL))
    AS weighted_utilization_sd_6m,

MAX(IF(window_months = 3,
    weighted_utilization_sd, NULL))
    AS weighted_utilization_sd_3m,


/* ========================================================
   REVOLVING UTILIZATION
   ======================================================== */

MAX(IF(window_months = 24,
    revolving_utilization_mean, NULL))
    AS revolving_utilization_mean_24m,

MAX(IF(window_months = 12,
    revolving_utilization_mean, NULL))
    AS revolving_utilization_mean_12m,

MAX(IF(window_months = 6,
    revolving_utilization_mean, NULL))
    AS revolving_utilization_mean_6m,
```

```
MAX(IF(window_months = 3,
    revolving_utilization_mean, NULL))
    AS revolving_utilization_mean_3m,


/* ============================================================
   DETERIORATION
   ============================================================ */

MAX(IF(window_months = 24,
    deterioration_count_total, NULL))
    AS deterioration_count_total_24m,

MAX(IF(window_months = 12,
    deterioration_count_total, NULL))
    AS deterioration_count_total_12m,

MAX(IF(window_months = 6,
    deterioration_count_total, NULL))
    AS deterioration_count_total_6m,

MAX(IF(window_months = 3,
    deterioration_count_total, NULL))
    AS deterioration_count_total_3m,


MAX(IF(window_months = 24,
    deterioration_count_avg_per_month, NULL))
    AS deterioration_avg_24m,

MAX(IF(window_months = 12,
    deterioration_count_avg_per_month, NULL))
    AS deterioration_avg_12m,

MAX(IF(window_months = 6,
    deterioration_count_avg_per_month, NULL))
    AS deterioration_avg_6m,

MAX(IF(window_months = 3,
    deterioration_count_avg_per_month, NULL))
    AS deterioration_avg_3m,


/* ============================================================
   IMPROVEMENT
   ============================================================ */
```

```
MAX(IF(window_months = 24,
   improvement_count_total, NULL))
   AS improvement_count_total_24m,

MAX(IF(window_months = 12,
   improvement_count_total, NULL))
   AS improvement_count_total_12m,

MAX(IF(window_months = 6,
   improvement_count_total, NULL))
   AS improvement_count_total_6m,

MAX(IF(window_months = 3,
   improvement_count_total, NULL))
   AS improvement_count_total_3m,


/* ============================================================
  ADVERSE ENTRY
  ============================================================ */

MAX(IF(window_months = 24,
   adverse_entry_count_total, NULL))
   AS adverse_entry_count_total_24m,

MAX(IF(window_months = 12,
   adverse_entry_count_total, NULL))
   AS adverse_entry_count_total_12m,

MAX(IF(window_months = 6,
   adverse_entry_count_total, NULL))
   AS adverse_entry_count_total_6m,

MAX(IF(window_months = 3,
   adverse_entry_count_total, NULL))
   AS adverse_entry_count_total_3m,


/* ============================================================
  ADVERSE RESOLVED
  ============================================================ */

MAX(IF(window_months = 24,
   adverse_resolved_count_total, NULL))
   AS adverse_resolved_count_total_24m,

MAX(IF(window_months = 12,
   adverse_resolved_count_total, NULL))
```

```
    AS adverse_resolved_count_total_12m,

MAX(IF(window_months = 6,
    adverse_resolved_count_total, NULL))
    AS adverse_resolved_count_total_6m,

MAX(IF(window_months = 3,
    adverse_resolved_count_total, NULL))
    AS adverse_resolved_count_total_3m,


/* ============================================================
   BECAME UNRATED
   ============================================================ */

MAX(IF(window_months = 24,
    became_unrated_count_total, NULL))
    AS became_unrated_count_total_24m,

MAX(IF(window_months = 12,
    became_unrated_count_total, NULL))
    AS became_unrated_count_total_12m,

MAX(IF(window_months = 6,
    became_unrated_count_total, NULL))
    AS became_unrated_count_total_6m,

MAX(IF(window_months = 3,
    became_unrated_count_total, NULL))
    AS became_unrated_count_total_3m,


/* ============================================================
   LEFT UNRATED
   ============================================================ */

MAX(IF(window_months = 24,
    left_unrated_count_total, NULL))
    AS left_unrated_count_total_24m,

MAX(IF(window_months = 12,
    left_unrated_count_total, NULL))
    AS left_unrated_count_total_12m,

MAX(IF(window_months = 6,
    left_unrated_count_total, NULL))
    AS left_unrated_count_total_6m,
```

```
MAX(IF(window_months = 3,
   left_unrated_count_total, NULL))
   AS left_unrated_count_total_3m,


/* ============================================================
  EFFECTIVE UPDATES
  ============================================================ */

MAX(IF(window_months = 24,
   effectively_updated_accounts_total, NULL))
   AS effectively_updated_accounts_total_24m,

MAX(IF(window_months = 12,
   effectively_updated_accounts_total, NULL))
   AS effectively_updated_accounts_total_12m,

MAX(IF(window_months = 6,
   effectively_updated_accounts_total, NULL))
   AS effectively_updated_accounts_total_6m,

MAX(IF(window_months = 3,
   effectively_updated_accounts_total, NULL))
   AS effectively_updated_accounts_total_3m,


/* ============================================================
  MAX DPD JUMP
  ============================================================ */

MAX(IF(window_months = 24,
   max_dpd_jump_window, NULL))
   AS max_dpd_jump_24m,

MAX(IF(window_months = 12,
   max_dpd_jump_window, NULL))
   AS max_dpd_jump_12m,

MAX(IF(window_months = 6,
   max_dpd_jump_window, NULL))
   AS max_dpd_jump_6m,

MAX(IF(window_months = 3,
   max_dpd_jump_window, NULL))
   AS max_dpd_jump_3m,


/* ============================================================
```

```
    COVERAGE
    ============================================================ */

  MAX(IF(window_months = 24,
    months_with_active_accounts, NULL))
    AS months_with_active_accounts_24m,

  MAX(IF(window_months = 12,
    months_with_active_accounts, NULL))
    AS months_with_active_accounts_12m,

  MAX(IF(window_months = 6,
    months_with_active_accounts, NULL))
    AS months_with_active_accounts_6m,

  MAX(IF(window_months = 3,
    months_with_active_accounts, NULL))
    AS months_with_active_accounts_3m


FROM
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_window_summary`

GROUP BY
  party_id
"

DBI::dbExecute(con, sql_step8)
```